

Rapport de Projet

Projet 4 : Analyseur de Grammaires Régulières

Cours : Théorie des Langages et Compilation

Réalisé par :

Diallo Sidy Mohamed Salim
Kebe Mohamed

Encadré par :

Kabbaj Adil

Date : Décembre 2024

Résumé

Objectif : Implémenter un programme qui utilise la grammaire régulière selon la notation théorique pour effectuer l'analyse des séquences en entrée (via le processus de dérivation).

Table des matières

1	Introduction	2
2	Analyse Théorique	2
2.1	Grammaire Régulière	2
2.2	Processus de Dérivation	2
2.3	Représentation par Automate	2
3	Architecture du Système	3
3.1	Le Moteur Logique (Backend)	3
3.1.1	Structures de Données (<code>config/graph.h</code>)	3
3.1.2	Opérations sur le Graphe (<code>config/graph.c</code>)	3
3.2	Le Parseur (<code>config/utils.c</code>)	5
3.2.1	Parsing de Fichier	5
3.2.2	Parsing de Texte (Saisie Manuelle)	8
3.2.3	Algorithme de Vérification de Chaîne	10
3.3	L'Interface Utilisateur (Frontend)	11
4	Manuel d'Utilisation	11
4.1	Installation des Dépendances	11
4.2	Compilation	11
4.3	Format de Grammaire	11
4.4	Exemple de Test Complet	11
4.5	Utilisation de l'Interface Graphique	12
5	Difficultés Rencontrées et Solutions	12
5.1	Gestion de la Mémoire en C	12
5.2	Algorithme de Vérification	13
6	Tests et Validation	13
6.1	Tests Unitaires (Version Console)	13
7	Conclusion	14
A	Structure du Projet	14
B	Exemple de Fichier de Grammaire	14
C	Source Code :	15
D	Références Bibliographiques	15

1 Introduction

Ce projet consiste en la conception et la réalisation d'un outil permettant de manipuler des **grammaires régulières**. L'enjeu est de transformer une définition textuelle de règles de production (ex : $A \rightarrow aB$) en une structure de données exploitable (un automate fini) afin de valider si des chaînes de caractères appartiennent au langage défini.

Le projet propose deux interfaces :

- Une **interface console** pour des tests rapides
- Une **interface graphique GTK+3** permettant une interaction intuitive avec visualisation en temps réel

2 Analyse Théorique

Le programme repose sur les concepts fondamentaux de la hiérarchie de Chomsky.

2.1 Grammaire Régulière

Les grammaires régulières sont des grammaires dont les règles de production sont de la forme :

- $A \rightarrow aB$
 - $A \rightarrow a$
 - $A \rightarrow \varepsilon$ (production epsilon, notée $\backslash \mathfrak{t}$ dans notre implémentation)
- où A et B sont des **non-terminaux** et a est un **symbole terminal**.

2.2 Processus de Dérivation

Pour valider une chaîne, le programme suit le cheminement des non-terminaux :

1. Partir de l'état initial (premier non-terminal)
2. Pour chaque symbole de la chaîne d'entrée, suivre la transition correspondante
3. Si après lecture complète de la chaîne nous atteignons un état marqué comme **final**, la séquence est **acceptée**

2.3 Représentation par Automate

La grammaire est stockée sous forme de **Graphe Orienté** où :

- Chaque **nœud** représente un état (non-terminal)
- Chaque **arc** représente une transition étiquetée par un symbole terminal
- Les états finaux correspondent aux non-terminaux ayant une production epsilon

Exemple de correspondance :

Grammaire	Automate
$A \rightarrow aB$	État $A \xrightarrow{a}$ État B
$B \rightarrow cB$	État $B \xrightarrow{c}$ État B
$B \rightarrow \varepsilon$	État B (final)

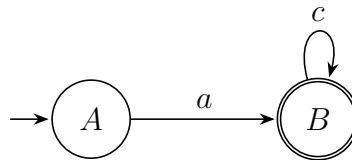


FIGURE 1 – Représentation graphique de l'automate

3 Architecture du Système

L'application est découpée en trois couches distinctes.

3.1 Le Moteur Logique (Backend)

Situé dans le dossier `config/`, il gère la structure de données.

3.1.1 Structures de Données (`config/graph.h`)

```

1 typedef char type;
2
3 // Structure representant une transition
4 typedef struct Children {
5     type value;           // Symbole de transition (terminal)
6     type transition;      // Etat cible
7     struct Children * next;
8 } children;
9
10 typedef children * Transitions;
11
12 // Structure representant un etat (noeud)
13 typedef struct Node{
14     type value;           // Identifiant de l'etat
15     int is_final;         // 1 si etat final, 0 sinon
16     Transitions transitions; // Liste des transitions sortantes
17     struct Node * next;
18 } node;
19
20 typedef node * list;

```

Listing 1 – Structures de données principales

Explication :

- La structure `children` forme une **liste chaînée de transitions** pour chaque état
- La structure `node` contient l'état lui-même et sa liste de transitions
- Le `list` est une **liste chaînée de tous les états** du graphe

3.1.2 Opérations sur le Graphe (`config/graph.c`)

Création et ajout de transitions :

```

1 void add_children(Transitions * transitions, type value, type transition
2 ){
3     // Allocation dynamique d'une nouvelle transition
4     children *c = (children*)malloc(sizeof(children));
5     if (!c) return;

```

```

5
6     c->transition = transition; // Etat cible
7     c->value = value;          // Symbole de transition
8     c->next = NULL;
9
10    // Insertion en fin de liste
11    if (*transitions == NULL) {
12        *transitions = c;
13    } else {
14        children *cur = *transitions;
15        while (cur->next != NULL) {
16            cur = cur->next;
17        }
18        cur->next = c;
19    }
20 }

```

Listing 2 – Fonction d’ajout de transition

Création et ajout d’un état :

```

1 void add_node_to_liste(list* l, Transitions transitions, type value, int
  is_final){
2     node *n = (node*)malloc(sizeof(node));
3     if (!n) return;
4
5     n->is_final = is_final;
6     n->value = value;
7     n->transitions = transitions;
8     n->next = NULL;
9
10    // Insertion en fin de liste
11    if (*l == NULL) {
12        *l = n;
13    } else {
14        node *cur = *l;
15        while (cur->next != NULL) {
16            cur = cur->next;
17        }
18        cur->next = n;
19    }
20 }

```

Listing 3 – Fonction d’ajout d’état

Recherche de transition :

```

1 children * get_transition(node n, type value){
2     Transitions t = n.transitions;
3     while (t != NULL) {
4         if (t->value == value) { // Symbole trouve
5             return t;
6         }
7         t = t->next;
8     }
9     return NULL; // Pas de transition pour ce symbole
10 }

```

Listing 4 – Fonction de recherche de transition

Recherche d’un état :

```

1 node * get_node(list l, type value){
2     node *cur = l;
3     while (cur != NULL) {
4         if (cur->value == value) {
5             return cur;
6         }
7         cur = cur->next;
8     }
9     return NULL;
10 }

```

Listing 5 – Fonction de recherche d'état

Affichage du graphe (console) :

```

1 void afficher_liste(list l) {
2     node *cur = l;
3     while (cur != NULL) {
4         children *trans = cur->transitions;
5         printf("%c", cur->value);
6
7         // Marquer les etats finaux avec *
8         if (cur->is_final) {
9             printf(" *");
10        }
11        printf(" : ");
12
13        // Afficher toutes les transitions
14        while (trans != NULL) {
15            printf("->%c%c", trans->value, trans->transition);
16            if (trans->next != NULL) {
17                printf(" | ");
18            }
19            trans = trans->next;
20        }
21        printf("\n");
22        cur = cur->next;
23    }
24 }

```

Listing 6 – Fonction d'affichage du graphe

3.2 Le Parseur (config/utils.c)

Le programme propose **deux modes d'entrée** pour charger une grammaire.

3.2.1 Parsing de Fichier**Lecture du fichier :**

```

1 char ** get_grammaire_file(char * url){
2     FILE *file = fopen(url, "r");
3     if (file == NULL) {
4         printf("Error: Could not open file '%s'\n", url);
5         return NULL;
6     }
7

```

```
8 // Compter le nombre de lignes
9 int line_count = 0;
10 char ch;
11 while ((ch = fgetc(file)) != EOF) {
12     if (ch == '\n') {
13         line_count++;
14     }
15 }
16
17 // Verifier la derniere ligne
18 fseek(file, -1, SEEK_END);
19 ch = fgetc(file);
20 if (ch != '\n' && ch != EOF) {
21     line_count++;
22 }
23
24 // Allocation du tableau de lignes
25 char **lines = (char **)malloc((line_count + 1) * sizeof(char *));
26 if (lines == NULL) {
27     printf("Error: Memory allocation failed\n");
28     fclose(file);
29     return NULL;
30 }
31
32 // Lecture ligne par ligne
33 fseek(file, 0, SEEK_SET);
34 char buffer[1024];
35 int i = 0;
36 while (fgets(buffer, sizeof(buffer), file) != NULL && i < line_count
37 ) {
38     size_t len = strlen(buffer);
39
40     // Supprimer le '\n' final
41     if (len > 0 && buffer[len - 1] == '\n') {
42         buffer[len - 1] = '\0';
43         len--;
44     }
45
46     // Copier la ligne
47     lines[i] = (char *)malloc(len + 1);
48     if (lines[i] == NULL) {
49         for (int j = 0; j < i; j++) {
50             free(lines[j]);
51         }
52         free(lines);
53         fclose(file);
54         return NULL;
55     }
56
57     strcpy(lines[i], buffer);
58     i++;
59 }
60
61 lines[i] = NULL; // Marquer la fin du tableau
62 fclose(file);
63 return lines;
```

Listing 7 – Fonction de lecture de fichier

Construction du graphe depuis un fichier :

```

1 list get_grammaire_list(char * line_transitions_tab){
2     if (line_transitions_tab == NULL) {
3         return NULL;
4     }
5
6     list l = NULL;
7     char **lines = get_grammaire_file(line_transitions_tab);
8
9     if (lines == NULL) {
10        return NULL;
11    }
12
13    // Parser chaque ligne
14    for (int i = 0; lines[i] != NULL; i++) {
15        char *line = lines[i];
16        if (strlen(line) == 0) {
17            continue;
18        }
19
20        char state;
21        char transition_input;
22        char next_state;
23        int is_terminal = 0;
24
25        // Parser format: "A -> bB" ou "B -> \t"
26        int pos = 0;
27
28        // Extraire l'état source (avant "->")
29        while (line[pos] == ' ') pos++;
30        state = line[pos];
31        pos++;
32
33        // Chercher "->"
34        while (line[pos] != '-' && line[pos] != '\0') pos++;
35        if (line[pos] == '\0') continue;
36        pos += 2; // Sauter "->"
37
38        // Extraire la partie droite
39        while (line[pos] == ' ') pos++;
40
41        // Verifier si c'est une production epsilon (\t)
42        if (line[pos] == '\\ ' && line[pos + 1] == 't') {
43            is_terminal = 1;
44            transition_input = '\0';
45            next_state = '\0';
46        } else {
47            // Lire symbole de transition et état suivant
48            transition_input = line[pos];
49            pos++;
50
51            if (line[pos] != '\0' && line[pos] != ' ') {
52                next_state = line[pos];
53            } else {
54                next_state = '\0';

```



```

55     }
56 }
57
58 // Chercher ou creer le noeud pour cet etat
59 node *current_node = get_node(l, state);
60
61 if (current_node == NULL) {
62     Transitions trans = NULL;
63     if (!is_terminal) {
64         add_children(&trans, transition_input, next_state);
65     }
66     add_node_to_liste(&l, trans, state, is_terminal);
67 } else {
68     if (!is_terminal) {
69         add_children(&(current_node->transitions),
70                     transition_input, next_state);
71     }
72     if (is_terminal) {
73         current_node->is_final = 1;
74     }
75 }
76 }
77
78 // Liberation memoire
79 for (int i = 0; lines[i] != NULL; i++) {
80     free(lines[i]);
81 }
82 free(lines);
83
84 return l;
85 }

```

Listing 8 – Fonction de construction du graphe depuis fichier

3.2.2 Parsing de Texte (Saisie Manuelle)

```

1 list parse_grammaire_text(char * grammar_text){
2     if (grammar_text == NULL || strlen(grammar_text) == 0) {
3         return NULL;
4     }
5
6     list l = NULL;
7
8     // Copier le texte pour tokenisation
9     char *text_copy = strdup(grammar_text);
10    char *line = strtok(text_copy, "\n");
11
12    while (line != NULL) {
13        // Ignorer les lignes vides
14        if (strlen(line) == 0) {
15            line = strtok(NULL, "\n");
16            continue;
17        }
18
19        char state;
20        char transition_input;
21        char next_state;

```

```

22     int is_terminal = 0;
23
24     int pos = 0;
25     while (line[pos] == ' ') pos++;
26     if (line[pos] == '\\0') {
27         line = strtok(NULL, "\\n");
28         continue;
29     }
30     state = line[pos];
31     pos++;
32
33     while (line[pos] != '-' && line[pos] != '\\0') pos++;
34     if (line[pos] == '\\0') {
35         line = strtok(NULL, "\\n");
36         continue;
37     }
38     pos += 2;
39
40     while (line[pos] == ' ') pos++;
41
42     if (line[pos] == '\\\\' && line[pos + 1] == 't') {
43         is_terminal = 1;
44         transition_input = '\\0';
45         next_state = '\\0';
46     } else {
47         transition_input = line[pos];
48         pos++;
49
50         if (line[pos] != '\\0' && line[pos] != ' '
51             && line[pos] != '\\r') {
52             next_state = line[pos];
53         } else {
54             next_state = '\\0';
55         }
56     }
57
58     node *current_node = get_node(l, state);
59
60     if (current_node == NULL) {
61         Transitions trans = NULL;
62         if (!is_terminal) {
63             add_children(&trans, transition_input, next_state);
64         }
65         add_node_to_liste(&l, trans, state, is_terminal);
66     } else {
67         if (!is_terminal) {
68             add_children(&(current_node->transitions),
69                         transition_input, next_state);
70         }
71         if (is_terminal) {
72             current_node->is_final = 1;
73         }
74     }
75
76     line = strtok(NULL, "\\n");
77 }
78
79 free(text_copy);

```

```

80     return 1;
81 }

```

Listing 9 – Fonction de parsing de texte

3.2.3 Algorithme de Vérification de Chaîne

Fonction principale de validation :

```

1  int check_string(char * ch, list l){
2      int n = strlen(ch);
3      int cpt = 0;
4
5      if (n == 0) return 0;
6      if (l == NULL) return 0;
7
8      // Commencer au premier etat (etat initial)
9      node * current = l;
10
11     // Parcourir chaque caractere de la chaine
12     for (int i = 0; ch[i] != '\0'; i++) {
13         cpt++;
14
15         // Chercher une transition pour ce caractere
16         children * transition = get_transition(*current, ch[i]);
17
18         if (transition == NULL) {
19             // Pas de transition : verifier si etat final
20             if (current->is_final == 1 && cpt == n - 1) {
21                 return 1; // Acceptee
22             }
23             return 0; // Rejetee
24         }
25
26         // Passer a l'etat suivant
27         node * next = get_node(l, transition->transition);
28         if (next == NULL) {
29             return 0; // Etat suivant introuvable
30         }
31         current = next;
32     }
33
34     // Après lecture complete : verifier si etat final
35     return current->is_final;
36 }

```

Listing 10 – Fonction de vérification de chaîne

Principe de l'algorithme :

1. Initialiser à l'état de départ (premier nœud)
2. Pour chaque symbole de la chaîne :
 - Chercher une transition correspondante
 - Si aucune transition n'existe : rejeter (sauf si état final)
 - Sinon, suivre la transition vers l'état suivant
3. À la fin : accepter ssi on est dans un état final

3.3 L'Interface Utilisateur (Frontend)

Développée en GTK+3, elle permet une interaction intuitive.

4 Manuel d'Utilisation

4.1 Installation des Dépendances

```
1 sudo apt-get install build-essential libgtk-3-dev
```

Listing 11 – Installation des dépendances

4.2 Compilation

Le projet utilise un **Makefile** pour automatiser la gestion des dépendances.

Commandes :

```
1 make          # Compile tout le projet
2 make main     # Compile uniquement la version console
3 make grammar_app # Compile uniquement l'interface graphique
4 make run      # Compile et execute la version console
5 make run-gui  # Compile et execute l'interface graphique
6 make clean    # Nettoie les fichiers compiles
```

Listing 12 – Commandes de compilation

4.3 Format de Grammaire

Les fichiers de grammaire (dans `input/`) utilisent le format :

```
1 A -> aB
2 B -> bB
3 B -> \t
```

Listing 13 – Format de fichier grammaire

Signification :

- A -> aB : Depuis l'état A, avec l'entrée 'a', transition vers l'état B
- B -> \t : L'état B est un état terminal (acceptant)

4.4 Exemple de Test Complet

Fichier de grammaire (`input/grammaire.txt`) :

```
1 A -> aB
2 B -> bB
3 B -> cB
4 B -> dB
5 B -> \t
```

Listing 14 – Exemple de grammaire

Langage accepté : $L = \{a(b|c|d)^*\}$

Tableau de tests :

Chaîne	Résultat	Explication
a	Acceptée	$A \rightarrow aB$, B est final
ab	Acceptée	$A \rightarrow aB \rightarrow bB$ (final)
abbb	Acceptée	$A \rightarrow aB \rightarrow bB \rightarrow bB \rightarrow bB$ (final)
acd	Acceptée	$A \rightarrow aB \rightarrow cB \rightarrow dB$ (final)
ba	Rejetée	Pas de transition 'b' depuis A
abc	Acceptée	$A \rightarrow aB \rightarrow bB \rightarrow cB$ (final)
aaa	Rejetée	Après 'aa', pas de transition 'a' depuis B
abcd	Acceptée	$A \rightarrow aB \rightarrow bB \rightarrow cB \rightarrow dB$ (final)

TABLE 1 – Tableau des résultats de tests

4.5 Utilisation de l'Interface Graphique

Étapes :

1. Lancer l'application :

```
1 make run-gui
2
```

2. Chargement de la grammaire (2 méthodes) :

Méthode 1 - Depuis un fichier :

- Onglet "Load from File"
- Saisir le chemin du fichier (défaut : `input/grammaire.txt`)
- Cliquer sur "Load Grammar"

Méthode 2 - Saisie manuelle :

- Onglet "Manual Input"
- Saisir les règles ligne par ligne
- Cliquer sur "Parse Grammar"

3. Visualisation :

- La grammaire s'affiche avec le format : État : $\rightarrow$ symboleÉtat_suivant
- Les états finaux sont marqués avec *

4. Test de chaînes :

- Saisir une chaîne dans le champ "Test String"
- Cliquer sur "Check String"
- Le résultat apparaît en **vert** (acceptée) ou **rouge** (rejetée)

5 Difficultés Rencontrées et Solutions

5.1 Gestion de la Mémoire en C

Problème : Les structures chaînées (`list`, `Transitions`) nécessitent une allocation dynamique avec `malloc()`. Risque de fuites mémoire si les pointeurs ne sont pas correctement libérés.

Solution :

- Vérification systématique du retour de `malloc()` :

```

1 children *c = (children*)malloc(sizeof(children));
2 if (!c) return; // Gestion d'erreur
3

```

— Libération explicite de la mémoire dans `get_grammaire_list()` :

```

1 for (int i = 0; lines[i] != NULL; i++) {
2     free(lines[i]);
3 }
4 free(lines);
5

```

5.2 Algorithme de Vérification

Problème : Gérer correctement le cas où la chaîne peut être acceptée avant d’avoir consommé tous les caractères (si l’état courant est final).

Solution : Ajout d’une condition spéciale :

```

1 if (transition == NULL) {
2     if (current->is_final == 1 && cpt == n - 1) {
3         return 1; // Accepter si etat final et un seul caractere
4         restant
5     }
6     return 0;
7 }

```

6 Tests et Validation

6.1 Tests Unitaires (Version Console)

Fichier de test (`main.c`) :

```

1 #include <stdio.h>
2 #include "config/graph.h"
3 #include "config/utils.h"
4
5 int main(){
6     list l = get_grammaire_list("input/grammaire.txt");
7
8     if (l != NULL) {
9         printf("Grammar loaded successfully:\n");
10        afficher_liste(l);
11    } else {
12        printf("Failed to load grammar\n");
13    }
14
15    // Tests
16    printf("result : %d\n", check_string("aaa", l)); // 0 (rejetee)
17    printf("result : %d\n", check_string("dddcccdcc", l)); // Selon
18    grammaire
19
20    return 0;
21 }

```

Listing 15 – Programme de test principal

7 Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux de la **théorie des automates** et des **grammaires formelles**. L'implémentation d'une interface graphique au-dessus d'un moteur de calcul en C démontre :

1. **La faisabilité d'outils pédagogiques** pour l'apprentissage de la compilation
2. **L'importance de la séparation des responsabilités** (Backend/Frontend)
3. **La puissance des structures de données** (graphes, listes chaînées) pour modéliser des concepts théoriques

Perspectives d'Amélioration

- **Optimisation mémoire** : Implémentation d'un garbage collector ou passage à C++
- **Extension aux grammaires context-free** : Support des grammaires de type 2
- **Visualisation graphique** : Affichage du graphe avec Cairo ou Graphviz
- **Export de résultats** : Génération de rapports PDF
- **Déterminisation** : Conversion NFA $\rightarrow$ DFA pour optimisation

A Structure du Projet

```

1 .
2 |-- config/
3 |   |-- graph.h           # Structures de donnees
4 |   |-- graph.c           # Operations sur le graphe
5 |   |-- utils.h           # Interfaces parsing/validation
6 |   |-- utils.c           # Implementation parsing/validation
7 |   |-- constant.h        # Constantes du projet
8 |   '-- constant.c
9 |-- ui/
10 |   |-- grammar_ui.h      # Interface GTK
11 |   |-- grammar_ui.c      # Implementation UI
12 |   '-- start.c           # Point d'entree GUI
13 |-- input/
14 |   '-- grammaire.txt      # Fichier de grammaire de test
15 |-- main.c                # Version console
16 |-- Makefile              # Scripts de compilation
17 '-- CLAUDE.md              # Documentation technique

```

Listing 16 – Arborescence du projet

B Exemple de Fichier de Grammaire

Grammaire reconnaissant les identifiants (commence par lettre, suivi de lettres/-chiffres) :

```

1 I -> aA
2 I -> bA
3 A -> aA
4 A -> bA

```

```
5 A -> 0A
6 A -> 1A
7 A -> \t
```

Listing 17 – Grammaire pour identifiants

Langage : $L = \{(a|b)(a|b|0|1)^*\}$

Exemples :

- a, b, a0, b1a, ab01 → Acceptés
- 0a, 1b, 01 → Rejetés (ne commencent pas par lettre)

C Source Code :

Le code source est disponible sur [GitHub](#).

D Références Bibliographiques

1. **Introduction to Automata Theory, Languages, and Computation** - Hopcroft, Motwani, Ullman
2. **Compilers : Principles, Techniques, and Tools** - Aho, Lam, Sethi, Ullman (Dragon Book)
3. **GTK+ 3 Reference Manual** - Documentation officielle GNOME
4. **The C Programming Language** - Kernighan & Ritchie